Лабораторная работа №12: Программирование в командном процессоре ОС UNIX

САКО ЛАССИНЕ

Лабораторная работа №12: Программирование в командном процессоре ОС UNIX

Цель работы

Изучение основ программирования в оболочке Bash. Освоение написания скриптов для автоматизации задач.

Ход выполнения работы

1. **Скрипт резервного копирования (backup.sh)**
2. **Скрипт обработки аргументов (args.sh)**
3. **Скрипт-аналог команды ls (myls.sh)**
4. **Скрипт подсчёта файлов по расширению (count.sh)**

Результаты выполнения

Скрипт	Описание	Результат
`backup.sh`	Резервное копирование	□ Архив создан
`args.sh`	Обработка аргументов	□ 12 аргументов
`myls.sh`	Аналог ls	□ Список каталогов
`count.sh`	Подсчёт по расширению	□ 5 файлов .sh

Выводы

В ходе выполнения лабораторной работы были освоены: - Создание резервных копий с помощью `tar` - Обработка аргументов командной

```
lsako@sakolassine:~/labos/lab12$ cat > backup.sh << 'EOF'
#!/bin/bash
# Script de sauvegarde du fichier lui-même

BACKUP_DIR=~/backup
mkdir -p "$BACKUP_DIR"

# Copie du script avec archivage tar
tar -czf "$BACKUP_DIR/backup_$(date +%Y%m%d_%H%M%S).tar.gz" "$0"

echo "✅ Sauvegarde effectuée dans $BACKUP_DIR"
ls -la "$BACKUP_DIR"
EOF

chmod +x backup.sh
./backup.sh
✅ Sauvegarde effectuée dans /home/lsako/backup
total 12
drwxrwxr-x   2 lsako lsako 4096 avril 29 00:20 .
drwxr-x---+ 29 lsako lsako 4096 avril 29 00:20 ..
-rw-rw-r--   1 lsako lsako  323 avril 29 00:20 backup_20260429_002045.tar.gz
lsako@sakolassine:~/labos/lab12$
```

Figure 1: Скрипт резервного копирования

```
lsako@sakolassine:~/labos/lab12$ cat > args.sh << 'EOF'
#!/bin/bash
# Script affichant tous les arguments

echo "Nombre d'arguments : $#"
echo "Tous les arguments : $@"
echo "Liste des arguments :"

for arg in "$@"; do
    echo "  - $arg"
done
EOF

chmod +x args.sh
./args.sh arg1 arg2 arg3 arg4 arg5 arg6 arg7 arg8 arg9 arg10 arg11 arg12
Nombre d'arguments : 12
Tous les arguments : arg1 arg2 arg3 arg4 arg5 arg6 arg7 arg8 arg9 arg10 arg11 arg12
Liste des arguments :
  - arg1
  - arg2
  - arg3
  - arg4
  - arg5
  - arg6
  - arg7
  - arg8
  - arg9
  - arg10
  - arg11
  - arg12
lsako@sakolassine:~/labos/lab12$
```

Figure 2: Скрипт обработки аргументов

```
lsako@sakolassine:~/labos/lab12$ cat > myls.sh << 'EOF'
#!/bin/bash
# Script imitant la commande ls

DIR=${1:-.}  # Répertoire par défaut = courant

if [ ! -d "$DIR" ]; then
    echo "Erreur : $DIR n'est pas un répertoire"
    exit 1
fi

echo "Contenu de $DIR :"
echo ""

for file in "$DIR"/*; do
    if [ -f "$file" ]; then
        if [ -r "$file" ]; then echo -n "r"; else echo -n "-"; fi
        if [ -w "$file" ]; then echo -n "w"; else echo -n "-"; fi
        if [ -x "$file" ]; then echo -n "x"; else echo -n "-"; fi
        echo " $(basename "$file")"
    elif [ -d "$file" ]; then
        echo "d $(basename "$file")/"
    fi
done
EOF

chmod +x myls.sh
./myls.sh ~/labos
Contenu de /home/lsako/labos :

d lab01/
d lab02/
d lab03/
d lab05/
d lab06/
d lab07/
d lab08/
d lab09/
d lab10/
d lab11/
d lab12/
d lab13/
d lab14/
```

Figure 3: Скрипт myls.sh

```
lsako@sakolassine:~/labos/lab12$ cat > count.sh << 'EOF'
#!/bin/bash
# Compte les fichiers par extension

if [ $# -ne 2 ]; then
    echo "Usage: $0 <extension> <repertoire>"
    echo "Exemple: $0 .txt /home"
    exit 1
fi

EXT=$1
DIR=$2

if [ ! -d "$DIR" ]; then
    echo "Erreur : $DIR n'est pas un répertoire"
    exit 1
fi

count=$(find "$DIR" -type f -name "*$EXT" 2>/dev/null | wc -l)
echo "Nombre de fichiers avec l'extension $EXT dans $DIR : $count"
EOF

chmod +x count.sh
./count.sh .sh ~/labos
Nombre de fichiers avec l'extension .sh dans /home/lsako/labos : 5
lsako@sakolassine:~/labos/lab12$
```

Figure 4: Скрипт count.sh

строки ($#, $@, $*) - Использование циклов `for` и условных операторов `if` - Поиск файлов с помощью `find` - Проверка прав доступа (`-r`, `-w`, `-x`, `-f`, `-d`)

Контрольные вопросы

1. Объясните понятие командной оболочки. Приведите примеры командных оболочек. Чем они отличаются?

Командная оболочка (shell) — программа, обеспечивающая интерфейс между пользователем и ОС. Примеры: sh, bash, csh, zsh. Отличаются синтаксисом и функциями.

2. Что такое POSIX?

POSIX — набор стандартов для совместимости UNIX-подобных систем.

3. Как определяются переменные и массивы в языке программирования bash?

“‘bash name=“value” set -A array a b c echo ${array[0]}

4. Каково назначение операторов let и read?

- `let` — арифметические вычисления
- `read` — чтение ввода пользователя

5. Какие арифметические операции можно применять в языке программирования bash?

`+`, `-`, `*`, `/`, `%`, `**`, `+=`, `-=`, `*=`, `/=`, `%=`

6. Что означает операция `(( ))`?

Арифметические вычисления и выполнение выражений.

7. Какие стандартные имена переменных Вам известны?

`PATH`, `HOME`, `PS1`, `PS2`, `IFS`, `MAIL`, `TERM`, `LOGNAME`

8. Что такое метасимволы?

Символы со специальным значением: `*`, `?`, `[]`, `|`, `>`, `<`, `&`, `;`

9. Как экранировать метасимволы?

- `\` — экранирование одного символа
- `'...'` — экранирование строки
- `"..."` — частичное экранирование

10. Как создавать и запускать командные файлы?

“‘bash nano script.sh chmod +x script.sh ./script.sh

11. Как определяются функции в языке программирования bash?

“‘bash function name { commands }

12. Каким образом можно выяснить, является файл каталогом или обычным файлом?

- `[ -d "$file" ]` — проверка, является ли файл каталогом
- `[ -f "$file" ]` — проверка, является ли файл обычным файлом

13. Каково назначение команд set, typeset и unset?

- `set` — установка параметров
- `typeset` — объявление типа переменной
- `unset` — удаление переменной

14. Как передаются параметры в командные файлы?

“‘bash ./script.sh arg1 arg2 # $1, $2, $#, $@

15. Назовите специальные переменные языка bash и их назначение.

Переменная	Назначение
`$0`	Имя скрипта
`$1-$9`	Позиционные параметры
`$#`	Количество аргументов
`$@`	Все аргументы
`$?`	Код завершения
`$$`	PID процесса
`$!`	PID фонового процесса

Заключение

Лабораторная работа выполнена в полном объёме.